Abstractive Summarisation with a Transformer Written from Scratch

Group Assignment 2, Question 3

1 Setup

We train a Transformer encoder-decoder (Vaswani et al., 2017) on CNN/Daily Mail v3.0.0 to turn news articles into multi-sentence summaries. Attention, feed-forward, positional-encoding, encoder- and decoder-layer code is written from basic tensor operations; no pre-trained weights are used. Only the tokenizer is pre-trained.

Table 1: Setup of the reported run.

Data	CNN/DM 3.0.0: all 287,113 training pairs (6.02 epochs), 13,368 validation (early stopping, decoding choice), 11,490 test (final ROUGE only)
Tokenizer	pre-trained `bert-base-uncased` WordPiece, lowercased text, plus `[NL]` for sentence breaks; shared vocabulary 30,523; `[CLS]`/`[SEP]` = start/end
Truncation	article 400 tokens (cuts 88% of test articles), summary 128 tokens (cuts 3.5%)
Model	$d_{\text{model}} = 512$, 8 heads, 4+4 layers, $d_{\text{ff}} = 2048$, dropout 0.1, post-norm, tied embeddings, copy switch; 45.05M parameters
Training	teacher forcing; cross-entropy over non-padding tokens, label smoothing 0.1; Adam $(0.9, 0.98, 10^{-9})$, Noam schedule (warm-up 4,000); fp16; batch 64 on 2×T4; 3.25 h
Decoding	greedy (required) and greedy + trigram blocking (main, chosen on validation); max. 128 tokens

2 Data and preprocessing

We use the official splits; the model learns only from the training split, validation is used for early stopping and for choosing the decoding setting, and the test split is used once for the final scores. Preprocessing, in order: (1) articles and highlights are lowercased to match the uncased tokenizer; (2) the newline between highlight sentences becomes the special token `[NL]`, so the model learns where sentences end and its output can be split into lines for ROUGE-Lsum (references keep their original newlines); (3) WordPiece tokenisation with one vocabulary shared by articles and summaries; (4) truncation to 400 article and 128 summary tokens, with `[SEP]` kept at the end; (5) each batch is padded to its longest row (`[PAD]` = id 0). The decoder input is the summary shifted right (starting with `[CLS]`) and the target is the summary shifted left (ending with `[SEP]`).

Table 2: Token lengths (including `[CLS]`/`[SEP]`) before truncation, and the share of texts that are cut.

Split	Text	Median	90th pct	Max	Limit	Truncated
train	article	800	1,481	6,323	400	89.9%
train	summary	63	99	2,302	128	2.8%
validation	article	765	1,476	2,417	400	87.1%
validation	summary	69	111	1,897	128	4.5%
test	article	780	1,504	2,967	400	87.8%
test	summary	66	104	859	128	3.5%

Most articles are cut, but news writing puts the key facts first, so the first 400 tokens (about half of a median article) contain most of what the highlights cover. Summaries are rarely cut.

3 Method

Transformer. Sinusoidal positional encodings, $PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{\text{model}}})$ and $PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{\text{model}}})$, are added to embeddings scaled by $\sqrt{d_{\text{model}}}$. Multi-head attention computes $\text{softmax}(QK^\top/\sqrt{d_k} + M)V$ with $h = 8$, $d_k = 64$ and separate Q, K, V and output projections; the FFN is $\max(0, xW_1 + b_1)W_2 + b_2$. Encoder layers apply self-attention and FFN; decoder layers apply masked self-attention, cross-attention and FFN, each wrapped as $\text{LayerNorm}(x + \text{Dropout}(\text{Sublayer}(x)))$. Masks: source padding (encoder self-attention and cross-attention) and causal + target padding (decoder self-attention). A final linear layer, tied to the embeddings (one matrix for encoder input, decoder input and output), gives vocabulary logits. Only `nn.Linear`, `nn.Embedding`, `nn.LayerNorm` and `nn.Dropout` are used as building blocks; `nn.Transformer*`, `nn.MultiheadAttention`, `F.scaled_dot_product_attention` and Hugging Face model classes are not.

Copy switch. A first run without copying wrote fluent but unrelated text early on and later misspelled rare names, because every name and number had to come from the vocabulary softmax. Following See et al. (2017), a gate $p_{\text{gen}} = \sigma(w^\top [s_t; c_t] + b)$ mixes generation and copying: $P(w) = p_{\text{gen}} P_{\text{vocab}}(w) + (1 - p_{\text{gen}}) \sum_{i:x_i=w} a_{t,i}$, where a_t is the last decoder layer's cross-attention averaged over heads and c_t the attended context (1,025 extra parameters). The loss is $-\log P(\text{target})$ of the mixed distribution.

Checks before training. The notebook stops unless: no forbidden layers are used; changing future tokens leaves earlier outputs unchanged (causal mask); extra padding changes nothing and gets zero attention (source and target padding masks); padding gets no loss or gradient; and a small model memorises 8 real pairs exactly. All passed.

4 Training

The learning rate follows $lr = d_{\text{model}}^{-0.5} \min(step^{-0.5}, step \cdot warmup^{-1.5})$, peaking at 6.99×10^{-4} after 4,000 steps; gradients are clipped to norm 1.0. Teacher forcing feeds the gold summary shifted right, so a whole summary is trained in one parallel pass. Every 3,000 steps the plain (unsmoothed) cross-entropy is computed on all 13,368 validation pairs (patience 3 checks), and 256 fixed validation articles are decoded greedily to track ROUGE of free-running output, which would expose a broken causal mask (falling loss, useless summaries).

Training stopped at the 3.25 h budget after 27,000 steps (1,728,000 pairs, peak GPU memory 7.1 GB). Validation loss fell at every check to 2.244 (perplexity 9.4) at the last one, so the model was still improving; this best checkpoint was restored before testing. Validation ROUGE-1/2/Lsum rose from 25.5/7.9/23.7 at step 3,000 to 39.4/17.3/36.5 at the best checkpoint. Scoring the 256 monitored reference summaries against a *wrong* article (each batch rotated by one) raised the loss from 2.16 to 5.29 nats/token (3.27 vs 6.55 at the first check), so predictions depend on the article. The copy gate settled at $p_{\text{gen}} \approx 0.22$: about 78% of the probability mass comes from copying.

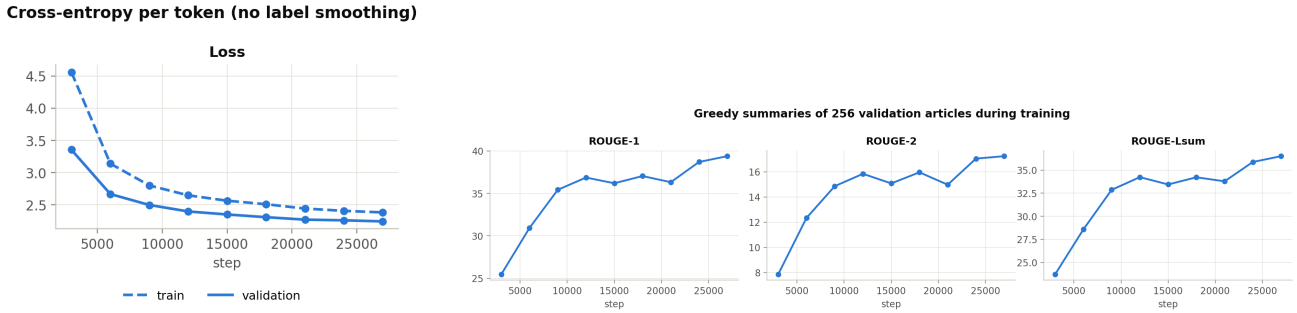


Figure 1: Left: per-token cross-entropy (training loss is measured with dropout on and averaged over each interval, hence above validation). Right: ROUGE of greedy summaries of 256 validation articles at each check.

5 Decoding

Generation uses no teacher forcing. Each article is encoded once; the decoder starts from `[CLS]` and appends the most likely token at each step until `[SEP]` or 128 tokens. `[PAD]`, `[CLS]` and `[UNK]` are never generated. Output is split into lines at `[NL]` before scoring. We also tried greedy decoding with *trigram blocking*, which forbids any token that would repeat a trigram already in the summary (Paulus et al., 2018). The choice was made on 1,000 validation articles, never on test:

Table 3: Decoding choice on 1,000 validation articles (ROUGE F1 ×100).

Decoding	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
Greedy	38.83	17.09	27.39	36.30
Greedy + trigram blocking	**40.25**	**17.91**	**27.74**	**37.81**

6 Results

Greedy + trigram blocking is the main setting; plain greedy, which the brief requires, is reported as well. Test scores use `rouge_score` with stemming, F1, on all 11,490 articles; our Lead-3 reproduces the brief's reference exactly, which confirms the scoring pipeline is the standard one.

Table 4: Test-set ROUGE F1 (×100) with 95% bootstrap confidence intervals. ROUGE-L treats a summary as one sequence; ROUGE-Lsum works sentence by sentence (split at newlines).

System	ROUGE-1	ROUGE-2	ROUGE-L	ROUGE-Lsum
Lead-3 (brief)	40.06	17.48	25.02	36.36
Lead-3 (our run)	40.05 [39.82, 40.28]	17.48 [17.27, 17.70]	25.02 [24.82, 25.22]	36.35 [36.13, 36.57]
Greedy	37.21 [36.98, 37.43]	15.83 [15.62, 16.05]	26.22 [26.02, 26.41]	34.65 [34.43, 34.87]
Greedy + trigram blocking	**38.42** [38.20, 38.63]	**16.38** [16.16, 16.60]	**26.32** [26.13, 26.53]	**35.93** [35.71, 36.13]

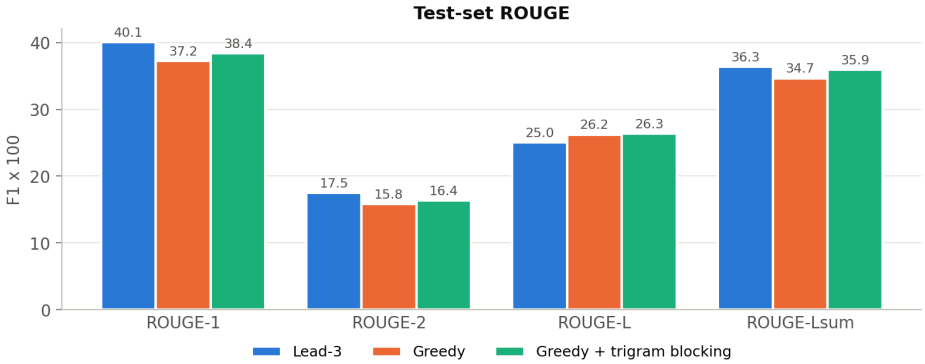


Figure 2: Test ROUGE of Lead-3 and the two decoding settings.

Comparison with Lead-3. The main model is 1.63 ROUGE-1, 1.10 ROUGE-2 and 0.42 ROUGE-Lsum below Lead-3, and 1.30 above it on ROUGE-L. The brief expects a from-scratch model to score below Lead-3, for several reasons. (i) *Lead bias:* journalists put the key facts first and the highlights paraphrase them, so Lead-3 gets facts, names and numbers exactly right at no cost, while our model must learn language, content selection and generation from random weights. (ii) *Small compute:* 6 epochs in 3.25 h with 45M parameters and no pre-training; published Transformer baselines train for days, and pre-trained models such as BART or PEGASUS add large-scale pre-training on top. (iii) *Greedy decoding* commits to one token at a time. (iv) *Truncation* hides facts from later in the article, which Lead-3 never uses anyway. (v) *ROUGE counts word overlap*, so a correct paraphrase scores below a copied sentence. ROUGE-L is lower than ROUGE-Lsum for every system because it needs one common subsequence across the whole summary, so content in a different sentence order is lost; ROUGE-Lsum matches each reference sentence separately.

Table 5: Properties of test summaries.

	Reference	Lead-3	Greedy	Greedy + blocking
Avg. words	52.9	78.8	56.0	55.4
Avg. sentences	3.88	3.00	3.54	3.59
With a repeated word trigram	7.2%	21.8%	57.3%	7.9%
With a number not in the article	9.8%	0%	0.6%	1.2%
Bigrams not in the article	50.0%	0%	9.8%	13.3%
Empty summaries	0%	0%	0%	0%

Plain greedy repeats a word trigram in 57% of summaries; trigram blocking brings this to 8% (it works on sub-word tokens and allows trigrams across a sentence break, so some word-level repeats remain) and adds 1.28 test ROUGE-Lsum. Copying keeps numbers faithful: about 1 summary in 100 has a number that the article does not contain, against 9.8% of human references, which sometimes compute or reword numbers. The check only compares digit strings, so it is rough. The model is mostly extractive (10-13% novel bigrams vs 50% for humans), which is why its numbers are rarely wrong but its wording stays close to the article's.

7 Qualitative analysis

Test examples were picked automatically by ROUGE and failure type; each claim was checked against the full article.

- **Highest (CNN Student News, ROUGE-Lsum 100 greedy / 97 blocked).** The highlights are standard boilerplate that also appears word for word in the article, so copying it scores perfectly. The score says more about this page than about summarisation skill; blocking slightly damages it ("cnn student cnn student call") because the correct text legitimately repeats "cnn student news".
- **Median (Robert Downey Jr, 34.1).** *"robert downey jr has been accused of making a racist remark. he is*

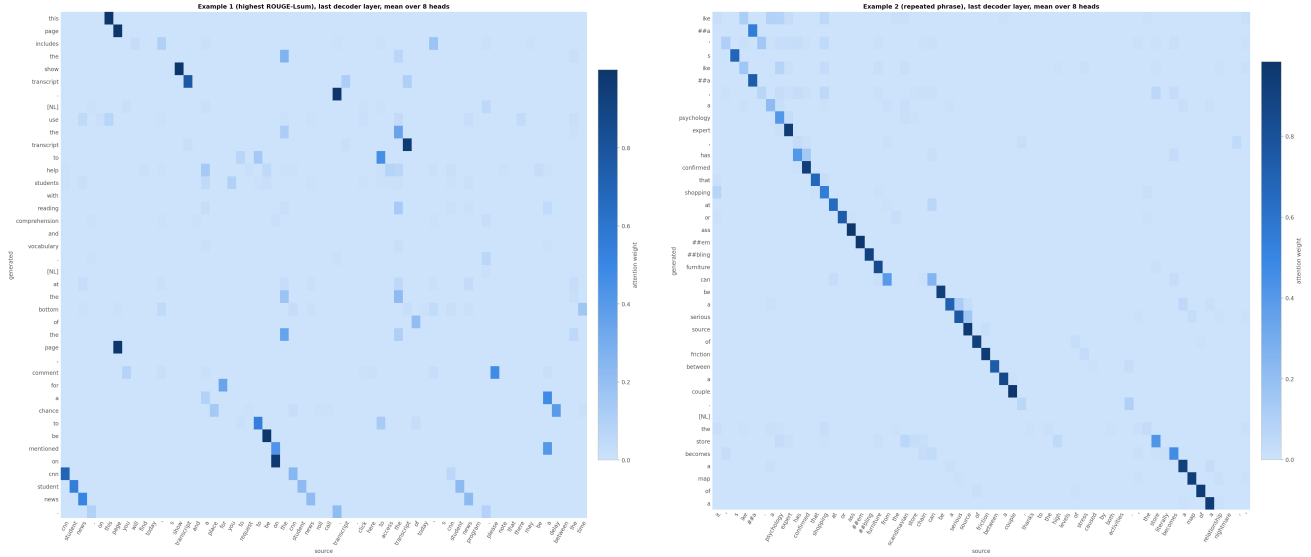
promoting his new film avengers: age of ultron. … he said: i think for a man whose native tongue is spanish to be able to put together a phrase like cultural genocide". Every sentence is in the article, but the summary is incomplete: it never names Iñárritu, *Birdman* or the TV walk-out, which come later. Greedy repeats "is promoting"; blocking only rewords it to "is promoting the film avengers and is promoting avengers", hiding the repetition without adding information.

- **Repeated phrase (IKEA, 33.0).** *"ikea's ikea, a psychology expert, has confirmed that shopping at or assembling furniture can be a serious source of friction between a couple."* The content is right, but the expert's name (Ramani Durvasula) is replaced by a copy of the article's opening "It's IKEA" (see Figure 3); the $1,199 Liatorp unit, half of the reference, is missed.
- **Number not in the article (Barcelona, 30.0).** *"barcelona defeated paris saint-germain 2-0 in the champions league. neymar has scored twice in the last 16 … luis enrique's side have now won just once in the last 16."* The first sentence is right; "twice", "16" and (with blocking) "four of their last four" are invented football statistics in the style of the training data.
- **Lowest (weekly trending-stories list, 0.0).** Every generated sentence comes from the article, but "is once again back in our lives, says mindy kaling's brother" joins two unrelated clauses into a false attribution. ROUGE is 0 because the reference covers other list items (the Burger-King couple, *The Vampire Diaries*); lead bias does not help on list articles.
- **Random (medical marijuana, 6.1).** *"lz granderson: i see signs of a revolution everywhere …"*. The copied text is correct, but "lz granderson" is a hallucinated author (a frequent CNN columnist) absent from the article, and the argument (legalise medical marijuana) is missed. The real author, Sanjay Gupta, appears only in the byline, so no article-only system could name him.

What the examples show. (1) Summaries are mostly faithful and extractive: sentences are copied or lightly shortened from the opening, matching the 10-13% novel bigrams and $p_{\text{gen}} = 0.22$. (2) Invented details appear where the model fills a template instead of copying: sports statistics, author names and "X says" attributions. They are fluent and plausible, so easy to miss without reading the article. (3) Content is missed when it is not near the start, or when a list article has many equally valid items. (4) Trigram blocking removes repeated phrases but not repeated content, and can break text that should repeat. (5) ROUGE rewards overlap with one reference, not correctness: the best example is boilerplate and the worst contains only article sentences.

8 Encoder-decoder attention

The heatmaps show the cross-attention of the last decoder layer, recorded during greedy generation (not teacher forcing) and averaged over the 8 heads; with the copy switch on, this is also the distribution the model copies from. Rows are generated tokens and columns are the 60-token article window that receives the most attention.



(a) CNN Student News example, article tokens 91-150. (b) IKEA example, article tokens 27-86.

Figure 3: Cross-attention recorded during greedy generation, last decoder layer, mean of 8 heads.

Copied phrases produce a sharp diagonal: each generated token puts almost all of its weight on the matching article token, and for 95% of generated words that occur in the article, the most-attended article token is that word, so attention acts as a soft pointer. The diagonal jumps to a new region after `[NL]`, when the model moves

to another part of the article. Function words at the start of a sentence (“the”, “a”) spread their weight more widely, because the next phrase has not been chosen yet. Attention also explains errors: “ikea’s ikea” is written while attending to “It’s IKEA.”, so the decoder copied the right words from the wrong place before moving on to “a psychology expert”.

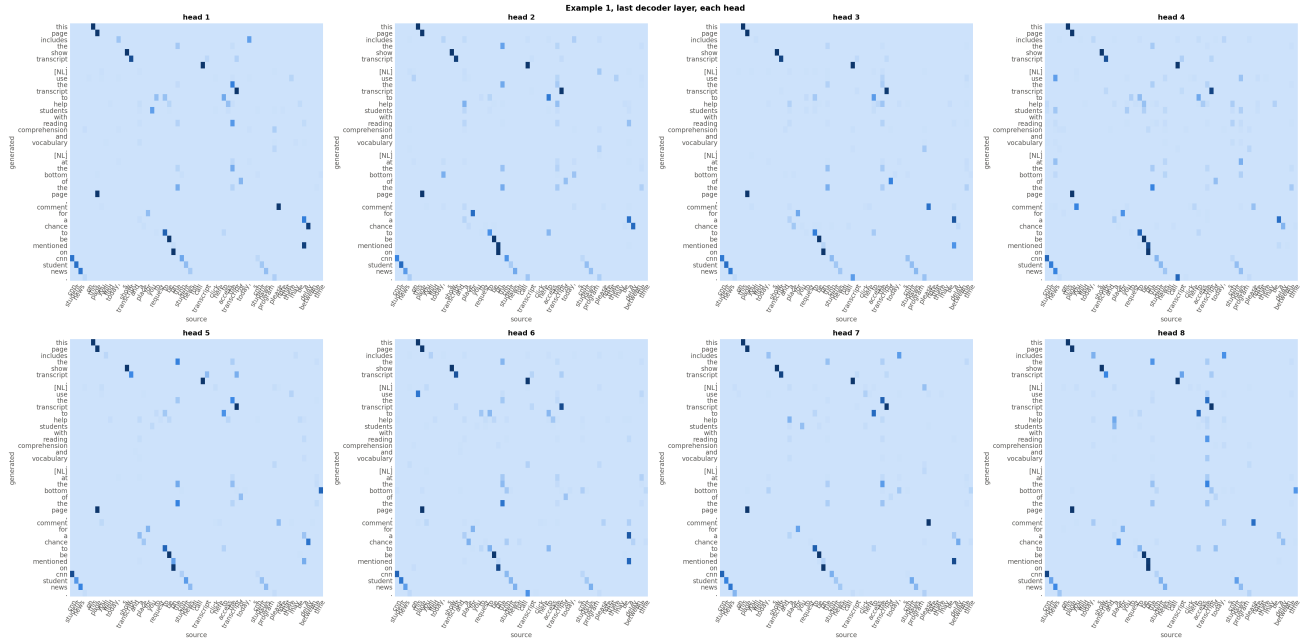


Figure 4: The CNN Student News example with each of the 8 heads of the last decoder layer shown separately.

The heads largely agree: all eight follow the same copy diagonal, so the head average used for copying is sharp. They differ on words that are not copied from this window (“help students with reading comprehension and vocabulary”), where most heads spread their weight thinly while head 8 keeps attending to a single token. Over 256 test summaries, 46% of the attention mass falls on the first 100 article tokens, which are 29% of the input (Figure 5): the decoder concentrates on the region Lead-3 copies, so it learned the lead bias of news from the data by itself. Attention shows where information was read from; it is not proof that the model understood it, and high attention on the right word does not guarantee a correct summary.

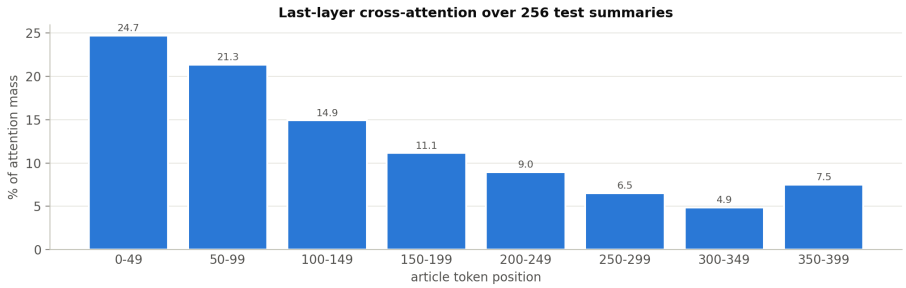


Figure 5: Cross-attention mass by article position (256 test summaries).